

Vidarbha Youth Welfare Society's
Prof. Ram Meghe Institute of Technology & Research,
Badnera-Amravati (M.S) 444701.



Laboratory Manual

Semester IV

Subject: - Data Structure and Algorithm Lab (4SX08)

Department of Computer Science and Engineering(IOT)

Vidarbha Youth Welfare Society's
Prof. Ram Meghe Institute of Technology & Research Badnera, Amravati
(M.S) 444701.



CERTIFICATE

This is to certify that Mr./Miss _____

Enrolment No. _____ Roll No. _____ Section _

Bachelor of Engineering, ***Second Year, Semester IV***, Department of Computer Science & Engineering(IOT) has satisfactorily completed the practical work of the subject **Data Structure and Algorithm Lab (4SX08)** prescribed by Prof. Ram Meghe Institute of Technology & Research, Badnera-Amravati (An Autonomous Institute) during the Academic Year 2025-26.

Practical No-01

Aim: - To study and execute the Linear search method.

Objective: Implementation of searching operations for a Key value in a given list of integers and analyze the complexities for the same.

Linear search

DATA – {11, 22, 33, 44, 55, 66, 77, 88, 99, 12}

KEY- {66}

Software Required: pycharm-community-2024-1.4

Theory:

a) Linear search:-

Linear search technique is also known as sequential search technique. Linear search is a method of searching an element in a list in sequence. In this method, the array is searched for the required element from the beginning of the list/array or from the last element to first element of array and continues until the item is found or the entire list/array has been searched.

Algorithm: (Linear Search) A linear array DATA with N elements and a specific ITEM of information are given. This algorithm finds the location LOC of ITEM in the array DATA or sets LOC: =0.

1. [Initialize] Set K: =1 and LOC: =0.
2. Repeat Steps 3 and 4 while LOC=0 and K<=N.
3. If ITEM=DATA [K], then: Set LOC: =K.
4. Set K: =K+1. [Increments counter.] [End of step 2 loop.]

5. [Successful.]

If LOC=0, then:

Write: ITEM is not in the array DATA Else:

Write: LOC is the location of ITEM. [End of If Structure.]

6. Exit.

Time complexity: Worst Case Analysis:

In the worst case analysis, we calculate the upper bound on running time of an algorithm. We must know the case that causes the maximum number of operations to be executed. For Linear Search, the worst case happens when the element to be searched (x in the above code) is not present in the array. When x

is not present, the search () function compares it with all the elements of arr[] one by one. Therefore, the worst case time complexity of linear search would be $O(n)$.

Average Case Analysis:

In average case analysis, we take all possible inputs and calculate computing time for all of the inputs. Sum all the calculated values and divide the sum by the total number of inputs. We must know (or predict) the distribution of cases. For the linear search problem, let us assume that all cases are uniformly distributed (including the case of x not being present in the array). So we sum all the cases and divide the sum by n. Following is the value of average case time complexity.

$$\begin{aligned}C(n) &= 1 * 1/n + 2 * 1/n + 3 * 1/n + \dots + n * 1/n \\&= (1 + 2 + 3 + \dots + n) * 1/n \\&= n * (n + 1) / 2 * 1/n \\&= (n + 1) / 2 \\&= O(n)\end{aligned}$$

Best Case Analysis:

In the best case analysis, we calculate a lower bound on the running time of an algorithm. We must know the case that causes the minimum number of operations to be executed. In the linear search problem, the best case occurs when x is present at the first location. The number of operations in the best case is constant (not dependent on n). So time complexity in the best case would be $O(1)$.

Program:

Linear Search Program

```
data = [11, 29, 35, 94, 55, 662, 77, 80, 90, 112, 54, 56, 78]
key = 54
```

```
def linear_search(data, key):
    for i in range(len(data)):
        if data[i] == key:
            return i + 1
    return 0
```

```
LOC = linear_search(data, key)
```

```
if LOC == 0:
    print("Key is not found in the list")
else:
    print("Key is found at location:", LOC)
```

Result & Discussion:

1. Conclusion

Practical No-02

Aim: - Understand the Linear data Structure and execute the Insertion, Deletion operation on it.

Objective: Implementation of Linear data Structure to carry out various operations on array like Insertion, Deletion with any unsorted array.

DATA={10,54,23,1,4,5,7,3,44,66}

Software Required: pycharm-community-2024-1.4

Theory:

Array:

An array data structure or simply an array is a data structure consisting of a collection of elements (values or variables), each identified by at least one array index or key. An array is stored so that the position of each element can be computed from its index tuple by a mathematical formula. Array types are often implemented by array structures; however, in some languages they may be implemented by hash tables, linked lists, search trees, or other data structures.

One-dimensional arrays

A one-dimensional array (or single dimension array) is a type of linear array. Accessing its elements involves a single subscript which can either represent a row or column index. As an example consider the C declaration `int an Array Name[10];`

Multidimensional arrays

For a two-dimensional array, the element with indices i, j would have address $B + c \cdot i + d \cdot j$, where the coefficients c and d are the row and column address increments, respectively.

More generally, in a k -dimensional array, the address of an element with indices $i_1, i_2,$

..., i_k is

$$B + c_1 \cdot i_1 + c_2 \cdot i_2 + \dots + c_k \cdot i_k.$$

For example: `int a[3][2];`

1. Algorithm:-

INSERT (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the K^{th} position in LA.

1. [Initialize Counter.] Set $J := N$.
2. Repeat Steps 3 and 4 while $J \geq K$.

3. [Move J^{th} element downward.] Set $LA [J+1] := LA [J]$.
4. [Decrease counter.] Set $J := J-1$. [End of Step 2 loop.]
5. [Insert element.] Set $LA [K] := \text{ITEM}$.
6. [Reset N .] Set $N := N+1$.
7. Exit.

2. DELETE ():

DELETE (LA, N, K, ITEM)

Here LA is a linear array with N elements and K is a positive integer such that $K \leq n$. This algorithm deletes the K^{th} elements from LA .

1. Set $\text{ITEM} := LA [K]$.
2. Repeat for $J = k$ to $N-1$:
 [Move $J + 1^{\text{st}}$ element upward.] Set $LA [J] := LA [J+1]$. [End of loop.]
3. [Reset the number N of elements in LA .] Set $N := N-1$. Exit.

Pseudo code:

Program: # Array Insertion

```
DATA = [20, 23, 77, 51, 23, 42]
N = 6
K = 5
ITEM = 71
```

```
J = N - 1
```

```
while J >= K - 1:
    DATA[J + 1] = DATA[J]
    J = J - 1
```

```
DATA[K - 1] = ITEM
N = N + 1
```

```
print("Updated DATA:", DATA[:N])
print("Updated N:", N)
```

Array Deletion

```
LA = [20, 23, 77, 51, 71]
N = 5
K = 3
```

```
ITEM = LA[K - 1]
```

```
for J in range(K - 1, N - 1):
```

$LA[J] = LA[J + 1]$

$N = N - 1$

```
print("Deleted Element:", ITEM)
print("Updated Array:", LA[:N])
print("Updated N:", N)
```

Result and Discussion:

Conclusion:

Practical No-03

Aim: To study and execute the Bubble Sort method.

Objective: Implement bubble sort for a given list of integers and display results after every pass give the time and space complexity for the same.

DATA – {98, 90, 12, 34, 20, 150, 54, 31, 86, 53}

Software Required: pycharm-community-2024-1.4

Theory:

Bubble Sort:-

Bubble sorting is a simple sorting technique in the sorting algorithm. In a bubble sorting algorithm, arrange the elements of the list by forming pairs of adjacent elements. It means repeatedly step through the list which we want to sort, compare two items at a time and swap them if they are not in the right order. Another way to visualize the bubble sort algorithm is as its name, the smaller element bubble to the top.

Steps:-

- Traverse a collection of elements:-
- Move from the front to the end
- Bubble” the largest value to the end using pairwise comparisons and swapping
- If we have N elements...And if each time we bubble an element, we place it in its correct location...Then we repeat the “bubble up” process $N - 1$ times.
- After passing 1 first largest element kept at last and the same other passes.

1. Complexity:-

1. Best Case performance – When the list is already sorted, we require only one pass to check, hence $O(n)$.
2. Worst Case performance – When the list is in the reverse order, n number of comparisons and swap are required to be done n number of times, hence $O(n^2)$.
3. Average Case performance – $O(n^2)$

4. It does not require any extra space for sorting, hence $O(1)$ extra space.

2. Algorithms:-

BUBBLE SORT (DATA, N)

Here DATA is an array with N elements. This algorithm sorts the elements in

DATA.

1. Repeat Steps 2 and 3 for K=1 to N-1.
2. Set PTR:=1. [Initializes pass pointer PTR]
3. Repeat while PTR<=N-K: [Executes pass.]
 - (a) If DATA [PTR]>DATA [PTR+1], then:
Interchanges DATA [PTR] and DATA [PTR+1]
[End of If structure.]
 - (b) Set
PTR:=PTR+ [End of inner loop.][End Step1 of outer loop.]
4. Exit.

Program:DATA = [22, 11, 44, 66, 33, 55, 88, 77]

N = len(DATA)

print("Original List:", DATA)

for K in range(0, N):

swapped = False

for PTR in range(0, N - K - 1):

if DATA[PTR] > DATA[PTR + 1]:

DATA[PTR], DATA[PTR + 1] = DATA[PTR + 1], DATA[PTR]

swapped = True

print("After pass", K + 1, ":", DATA)

if not swapped:

print("If not swapped:")

break

Result and Discussion:

Conclusion:

Practical No-04

Aim: To study and execute the Binary Search method.

Objective: Implement the searching in a given list of integers with Key value and analyze the complexities for the same.

Binary search

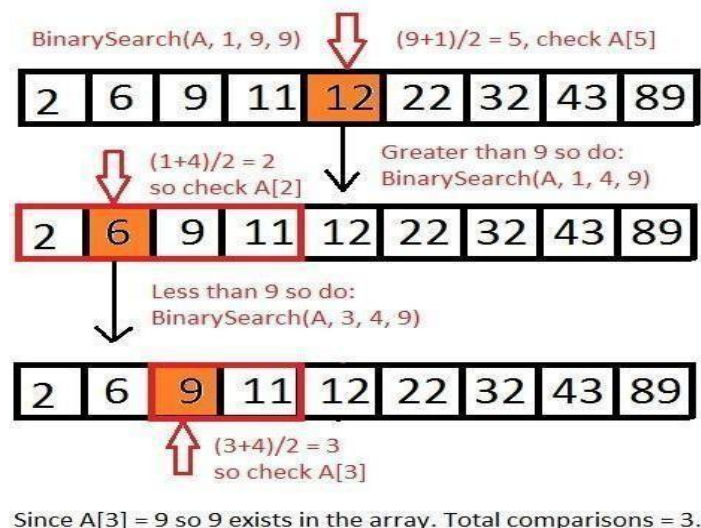
DATA – {10, 14, 27, 28, 40, 65, 68, 89, 98, 100}

KEY- {28}

Software required: pycharm-community-2024-1.4

Theory:

A binary search or half-interval search algorithm finds the position of a specified value (the input "key") within a sorted array. In each step, the algorithm compares the input key value with the key value of the middle element of the array. If the keys match, then a matching element has been found so its index, or position, is returned. Otherwise, if the sought key is less than the middle element's key, then the algorithm repeats its action on the sub-array to the left of the middle element or, if the input key is greater, on the sub-array to the right. If the remaining array to be searched is reduced to zero, then the key cannot be found in the array and a special "Not found" indication is returned. A binary search halves the number of items to check with each iteration, so locating an item (or determining its absence) takes logarithmic time. A binary search is a divide and conquer search algorithm.



Complexity Analysis of Binary Search:

Best Case: Best case occurs when the key is in the middle of the array. Runtime will require exactly one loop.

Worst Case: Key is not in the array. It works like reducing the size of array

to 1 by halving the array let m times.

$$n, n/2, n/2^2, n/2^m$$

$$n/2^m =$$

$$1$$

$$n = 2^m$$

$$\log_2 n = m$$

So, worst case complexity of binary search is $\log_2 n$. Only one drawback of binary search affects most of the time, which is, the array must be in sorted order

Algorithms: -

Binary Search (DATA, LB, UB, ITEM, LOC):

Here DATA is a sorted array with lower bound LB and upper bound UB and ITEM is a given item of information. The variables BEG, END and MID denote, respectively, the beginning, end and middle locations of a segment of elements of DATA. This algorithm finds the location LOC of ITEM in DATA or sets LOC=NULL.

1. [Initialize segment variables.]
Set BEG: = LB, END: = UB and MID=INT ((BEG+END)/2).
2. Repeat Steps 3 and 4 while BEG<=END AND DATA [MID]! =ITEM.
3. If ITEM< DATA[MID],
then: Set END:
=MID-1.
Else:
Set BEG: = MID+1.
[End of If statement.]
4. Set MID: = INT
((BEG+END)/2). [End of Step 2
loop].
5. If DATA[MID] = ITEM, then:
Set LOC: = MID.
Else:
Set LOC: =NULL.
[End of If structure.]
6. Exit.

```
PROGRAM DATA = [11, 22, 33, 44, 55, 66, 77]
ITEM = int(input("Enter item to search: "))
```

```
LB = 0
```

```
UB = len(DATA) - 1
```

```
LOC = 0
```

```
BEG = LB
```

```
END = UB
```

```
while BEG <= END:
    MID = (BEG + END) // 2

    if DATA[MID] == ITEM:
        LOC = MID + 1
        break
    elif ITEM < DATA[MID]:
        END = MID - 1
    else:
        BEG = MID + 1

if LOC == 0:
    print("Item not found")
else:
    print("Item found at location:", LOC)
```

Result and Discussion:

Conclusion:

Practical No-05

Aim: Discuss the singly linked list and execute Insertion, Deletion operation on it.

Objective:

- Create a node using Structures to implement a linked list of numeric numbers and display the linked list.
- In the above list insert a new node -
 - At first position on Linked List.
 - After a given node.
- Delete a node of a given data, if it is found, otherwise display appropriate messages and Display the contents of the list.

Software Required: pycharm-community-2024-1.4

Theory:

Linked List: -

A linked list is a linear data structure, in which the elements are not stored at contiguous memory locations. The elements in a linked list are linked using pointers.

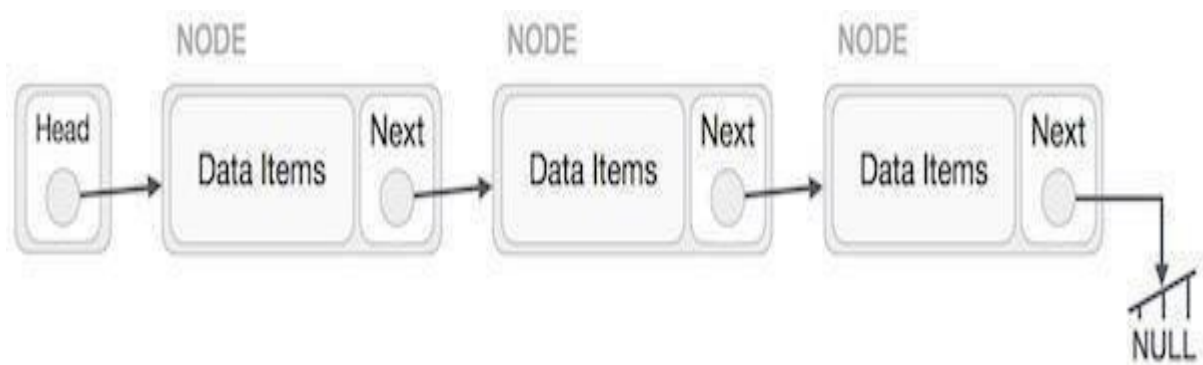


Fig: Linked List Representation

Linked list consists of nodes where each node contains a data field and a reference (link) to the next node in the list.

Basic Operations:-

- **Insertion** – Adds an element at the beginning of the list.
- **Deletion** – Deletes an element at the beginning of the list.
- **Display** – Displays the complete list.
- **Search** – Searches an element using the given key.

- **Delete** – Deletes an element using the given key.
- **Insertion:-**

A node can be added in three ways

- 1) At the front of the linked list
- 2) After a given node.
- 3) At the end of the linked list.

Algorithm:-

Inserting at the beginning of a list.

INSERT (INFO, LINK, START, AVAIL, ITEM)

This algorithm inserts ITEM as the first node in the list.

2. [OVERFLOW?] if
 AVAIL=NULL, then: Write:
 OVERFLOW, and Exit.

3. [Remove first node from AVAIL list.]

Set NEW: = AVAIL and AVAIL: = LINK
 [AVAIL].

4. Set INFO [NEW]:=ITEM. [Copies new data into new node].
5. Set LINK [NEW]:= START [New node now points to original first node.]
6. Set START:=NEW [Changes START so it points to the new node]
7. Exit.

Inserting after a given node

INSLOC (INFO, LINK, START, AVAIL, LOC, ITEM)

This algorithm insert ITEM so that ITEM follows the node with location LOC or insert ITEM as the first node when LOC=NULL.

1. [OVERFLOW?] if
 AVAIL=NULL, then: Write:
 OVERFLOW, and Exit.

2. [Remove first node from AVAIL List]

Set NEW: AVAIL and AVAIL:=LINK[AVAIL].

3. Set INFO[NEW]:= ITEM [Copies new data into new node.]

4. If LOC:=NULL, then: [Insert as first node]

Set LINK[NEW]:=START and START:=NEW

Else: [Insert after node with location
 LOC]

Set
 LINK[NEW]:=LINK[LOC] and

```
    LINK[LOC]:=NEW
[End of if structure]
5. Exit
```

Deletion:

To delete a node from linked list, the following steps are performed-

- 1) Find the previous node of the node to be deleted.
- 2) Change the next of the previous nodes.

Free memory for the node to be deleted

Algorithm:-

DEL (INFO, LINK, START, AVAIL, LOC, LOCP)

This algorithm deletes the node N with location LOC. LOCP is the location of the node which precedes N or when N is the first node LOCP=NULL.

1. If LOCP:=NULL, then:
 Set START:=LINK [START] [Deletes first node] Else
 Set LINK [LOCP]:= LINK [LOC]. [Deletes node N] [End of If structure].
2. [Return deleted node to the AVAIL [LIST] Set LINK[LOC]:=AVAIL and
 AVAIL:= LOC
3. Exit

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = None

def insert_begin(item):
    global head
    new = Node(item)
    new.next = head
    head = new

def insert_after(loc, item):
    temp = head
    while temp:
        if temp.data == loc:
            new = Node(item)
            new.next = temp.next
```



```
        temp.next = new
        return
    temp = temp.next
    print("Node not found")
```

```
def delete(item):
    global head
    temp = head
    prev = None

    while temp:
        if temp.data == item:
            if prev is None:
                head = temp.next
            else:
                prev.next = temp.next
            return
        prev = temp
        temp = temp.next

    print("Data not found")
```

```
def display():
    temp = head
    while temp:
        print(temp.data, end=" -> ")
        temp = temp.next
    print("None")
```

```
insert_begin(30)
insert_begin(20)
insert_begin(10)
```

```
print("Linked List:")
display()
```

```
insert_after(20, 25)
print("After insertion:")
display()
```

```
delete(30)
print("After deletion:")
display()
```

Result and Discussion:

Conclusion:

Questions:

1. What is a linked list?
2. What is a Singly Linked List?
3. State applications of singly linked list?
4. Give the importance of linked list and pointers in linked list?

Practical No-06

Aim: Define the stack structure and execute the push, pop operation on it.

Objective: Construct a stack of integers and perform the following Operations on it.

- a. Push
- b. Pop
- c. Display

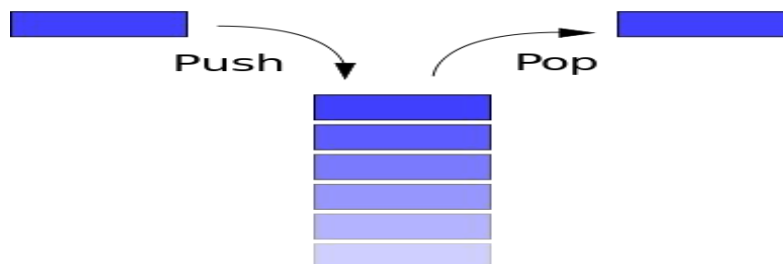
The program should print appropriate messages for stack overflow, stack underflow and stack empty.

Software required: pycharm-community-2024-1.4

Theory:

Stacks

A stack is a basic data structure, where insertion and deletion of items takes place at one end called top of the stack. This structure is used all throughout programming. The basic concept can be illustrated by thinking of your data as a stack of plates or books where you can only take the top item off the stack in order to remove things from it. A stack is also called a LIFO (Last In First Out) to demonstrate the way it accesses data. There are basically three operations that can be performed on stacks. They are 1) inserting an item into a stack (push). 2) Deleting an item from the stack (pop). 3) Displaying the contents of the stack (pip).



1. Application of stack:-

- a. Stacks are used to manage methods at runtime (when a method is called, its parameters and local variables are pushed onto a stack; when the method returns, the values are popped from the stack).
- b. Many parsing algorithms (used by compilers to determine whether a program is syntactically correct) involve the use of stacks.
- c. Stacks can be used to evaluate arithmetic expressions (e.g., by a simple calculator program), and they are also useful for some operations on **many** data structures.

2. Algorithms:-

Push (STACK,ITEM,TOP):

Description: Here STACK is an array with MAX locations. TOP points to the top most element and ITEM is the value to be inserted.

1. If (TOP == MAX) Then [Check for overflow]
2. Print: Overflow
3. Else
4. Set TOP = TOP + 1 [Increment TOP by 1]
5. Set STACK[TOP] = ITEM [Assign ITEM to top of STACK]
6. Print: ITEM
 inserted [End of If]
- Exit

Pop (STACK,ITEM,TOP):

Description: Here STACK is an array with MAX locations. TOP points to the top most element.

1. If (TOP == 0) Then [Check for underflow]
2. Print: Underflow
3. Else
4. Set ITEM = STACK[TOP] [Assign top of STACK to ITEM]
5. Set TOP = TOP - 1 [Decrement TOP by 1]
6. Print: ITEM
 deleted [End of If]
7. Exit.

Program:

MAX = 3

stack = []

def push(item):

 if len(stack) == MAX:

 print("Overflow")

 else:

 stack.append(item)

 print(item, "inserted")

def pop():

 if len(stack) == 0:

 print("Underflow")

 else:

 print(stack.pop(), "deleted")

def display():

 if len(stack) == 0:

 print("Stack is empty")

```
else:  
    print("Stack:", stack[::-1])
```

```
push(10)  
push(20)  
push(30)  
push(40)
```

```
display()
```

```
pop()  
display()
```

Result and Discussion:

Conclusion:

Questions:

1. What is stack?
2. Give real time example of stack?
3. What is meant by push and pop?
4. When overflow will occur in stack?

Practical No-07

Aim: Summarize the Queue structure and execute the insertion, deletion operation on it.

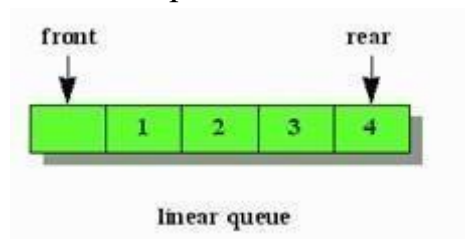
Objective: Simulate the working of a linear queue of integers using an array. Perform the following operations.

- a. Insert
- b. Delete
- c. Display

Software Required: pycharm-community-2024-1.4

Theory: A queue is an ordered collection of items where the addition of new items happens at one end, called the “rear,” and the removal of existing items occurs at the other end, commonly called the “front.” As an element enters the queue it starts at the rear and makes its way toward the front, waiting until that time when it is the next element to be removed. The most recently added item in the queue must wait at the end of the collection. The item that has been in the collection the longest is at the front. This ordering principle is sometimes called FIFO, first-in first-out. It is also known as “first-come first-served.”

Linear Queue: - A linear queue is like a straight line in which all elements or instructions stand one behind the other. There is a definite beginning and a definite end of the queue. Tasks lined up in this queue format are executed in the order of their placement, on a FIFO (First In First Out) basis. In a linear queue, a new task is inserted at the end of the list, while a deletion is made at the front of the list. Ex: A line for movie tickets and customers waiting at a gas station for their turn are examples of a linear queue structure.



A queue is structured, as described above, as an ordered collection of items which are added at one end, called the “rear,” and removed from the other end, called the “front.” Queues maintain a FIFO ordering property. The queue operations are given below.

Queue() creates a new queue that is empty. It needs no parameters and returns an empty queue.

enqueue(item) adds a new item to the rear of the queue. It needs the item and returns nothing.

dequeue() removes the front item from the queue. It needs no parameters and returns the item. The queue is modified.

isEmpty() tests to see whether the queue is empty. It needs no parameters and returns a Boolean value.

Size () returns the number of items in the queue. It needs no parameters and returns an integer.

Algorithms:-

QINSERT (QUEUE, N, FRONT, REAR, ITEM)

This procedure inserts an element ITEM into the queue.

1. [Queue is already filled?]
 If FRONT=1 and Rear=N or if
 Front=REAR+1, then: Write: Overflow,
 and Return.
2. [Find new value of REAR.]
 If FRONT: =NULL, then: [QUEUE is initially
 empty] Set FRONT: =1 and REAR: =1.
 Else if REAR=N,
 then:
 Set REAR: =1
 Else:
 Set REAR: =REAR+1.
 [End of if structure]
3. Set QUEUE [REAR]:=ITEM. [This inserts new element.]
4. Return.

QDELETE (QUEUE, N, FRONT, REAR, ITEM)

This procedure DELETES an element from queue and assigns it to variable ITEM .

1. [Queue is already
 filled?] If
 FRONT=NULL,
 then:
 Write: UNDERFLOW, and Return.
2. Set ITEM:=QUEUE[FRONT]
3. [Find new value of REAR.]
 If FRONT: =REAR, then: [QUEUE has only one
 element to start.] Set FRONT: =NULL and
 REAR: =NULL.
 Else if REAR=N, then:
 Set FRONT: =1

```
        Else:
            Set FRONT: =FRONT+1.
[END OF IF STRUCTURE]
4. Return.
```

Program:

```
class Queue:
    def __init__(self, n):
        self.n = n
        self.queue = []

    def insert(self, item):
        if len(self.queue) == self.n:
            print("Overflow")
        else:
            self.queue.append(item)
            print("Inserted:", item)

    def delete(self):
        if len(self.queue) == 0:
            print("Underflow")
        else:
            print("Deleted:", self.queue.pop(0))

    def display(self):
        if len(self.queue) == 0:
            print("Queue is empty")
        else:
            print("Current Queue:", end=" ")
            for i in self.queue:
                print(i, end=" ")
            print()

q = Queue(3)

q.insert(10)
q.insert(20)
q.insert(30)
q.insert(40)

q.display()

q.delete()
q.display()
```


Result and Discussion:**Conclusion:****Questions:**

- 1) What is a linear queue?
- 2) What is rear and front pointer?
- 3) Give real time example of queue?
- 4) State different applications of queue.

Practical No- 8

Aim: Discuss the Insertion sorting operation and implement on given DATA.

Objective: Implement Insertion sort for a given list of integers and display results after every pass also analyze time and space complexity for the same.

DATA – { 40,10,9,20,30,50}

Software required: pycharm-community-2024-1.4

Theory:

Insertion sort is a simple sorting algorithm that builds the final sorted array (or list) one item at a time. It is much less efficient on large lists than more advanced algorithms such as quick sort, heap sort, or merge sort. Insertion sort iterates, consuming one input element each repetition, and growing a sorted output list. At each iteration, insertion sort removes one element from the input data, finds the location it belongs within the sorted list, and inserts it there. Insertion sort is stable and has quadratic asymptotic complexity $O(n^2)$. If the input sequence is already sorted, then the insertion sort only checks the ordering and does not perform any moves. Hence if there are only few unsorted elements, the complexity of insertion sort gets close to $O(n)$.

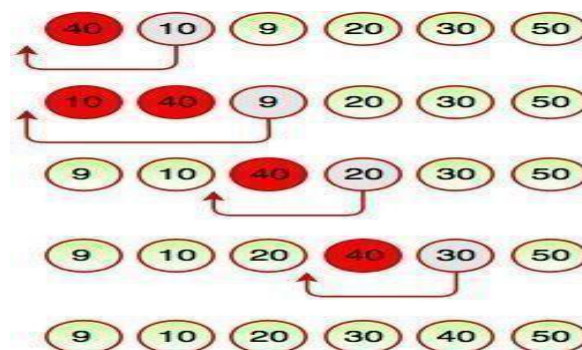


Fig. Working of Insertion Sort

Algorithm:

(Insertion Sort) **INSERTION (A,N)**

This algorithm sorts the array A with N elements-

1. Set $A[0] := -\infty$ [Initialize Sentinel element]
2. Repeat step 3 to 5 for $k=2,3,4\dots N$.
3. Set $Temp := A[k]$ and $PTR=k-1$;
4. Repeat while $TEMP < A[PTR]$:

- a) Set $A[PTR+1] := A[PTR]$. [Move element forward]
- b) Set $PTR := PTR - 1$ [End of loop]
- 5. Set $A[PTR+1] := TEMP$ [Inserts element in proper place] [End of Step 2 loop]
- 6. Return

Program:

```
DATA = [40, 10, 9, 20, 30, 50]
```

```
print("Initial Data:", DATA)
```

```
for i in range(1, len(DATA)):
```

```
    temp = DATA[i]
```

```
    j = i - 1
```

```
    while j >= 0 and DATA[j] > temp:
```

```
        DATA[j + 1] = DATA[j]
```

```
        j -= 1
```

```
    DATA[j + 1] = temp
```

```
    print("Pass", i, ":", DATA)
```

```
print("Final Sorted Result:", DATA)
```

Result and Discussion:

Conclusion:

Questions:

1. What is insertion sort? Explain its sorting technique?
2. What are the different types of sorts in data structures?
3. What is the efficiency of insertion sort in worst case?
4. What is the worst case time complexity of insertion sort

